domingo, 30 de noviembre de 2025

UNIVERSIDAD DE SONORA

Facultad Interdisciplinaria de Ingenierías



3er Semestre

Ingeniería en Sistemas de Información

FORNITE 2

Cordova Montiel Axel

Fajardo Sepulveda Kevin Jovanni

Salcido Peralta Jorge Manuel

García Horta Abril Melissa

Maestro: Jose Carlos Mercado Chan

Contenido

Introducción 3

Justificación 4

Descripción del problema 4

Objetivo del Proyecto 5

Actores que Interactúan con el Sistema 5

Funcionalidades del Sistema 5

Funciones principales 5

Funciones opcionales 6

Definir Requerimientos Funcionales y No Funcionales 6

Requerimientos Funcionales (Qué debe hacer el sistema) 6

Requerimientos No funcionales (Cómo debe funcionar el sistema) 6

Diagrama Base de datos 7

Diagrama UML 8

Diseño de las vistas 9

Ventana login 9

Ventana de habilidades 10

Ventana de armas 11

Ventana de clases 12

Ventana personaje 13

Ventana que relaciona nivel de la habilidad y el personaje 14

Introducción

Integrantes	Axel	Kevin	Jorge	Melissa
AXEL CORDOVA MONTIEL Un sistema de gestión de ventas de una empresa		0		0
ABRIL MELISSA GARCIA HORTA Un sistema de gestión de justificantes médicos en la unison	1	1	0	1
JORGE MANUEL SALCIDO PERALTA Expandir Fortnite 2 (juego rpg que hice de proyecto el semestre pasado) para implementar conexión con base de datos para los equipamentos, habilidades, enemigos y personajes.	1	1	1	
KEVIN YOVANNI FAJARDO SEPULVEDA Sistema donde se realiza una reseña de algún videojuego	0		1	1

Durante la primera etapa del proyecto, los integrantes del equipo propusieron diversas ideas (ver Tabla 1).

Luego de analizar las propuestas, el equipo seleccionó la opción de expandir *Fortnite 2* mediante el desarrollo de un sistema completo para administrar múltiples elementos del juego, tales como efectos de estado, objetos, movimientos, personajes, enemigos y equipamiento.

Sin embargo, tras revisar el alcance y la complejidad del proyecto, el equipo concluyó que esta idea era demasiado amplia para el tiempo y recursos disponibles. Por ello, se decidió realizar un **cambio de proyecto**, reduciendo el alcance a un sistema CRUD más sencillo y manejable.

El nuevo proyecto consiste en desarrollar un sistema para administrar atributos esenciales del juego, enfocado únicamente en Personajes no jugadores y sus características principales, tales como armas, habilidades y clases/roles.

Justificación

Tras analizar las propuestas, el equipo seleccionó la opción de desarrollar un Sistema CRUD para la gestión de Atributos de personajes no jugadores (NPC), ya que este tipo de sistema aborda un problema crítico en el proceso de desarrollo del juego: la administración manual y dispersa de la información.

Contar con un sistema centralizado permite estandarizar y organizar los datos, reducir errores manuales, evitar inconsistencias entre versiones e incrementar la eficiencia del equipo de diseño, que podrá modificar y consultar atributos de personajes, armas, habilidades, y clases de manera ágil y controlada.

Descripción del problema

La gestión manual de los atributos de los personajes no jugadores (NPC), como sus armas, habilidades, y clases, puede generar inconsistencias en los datos, duplicidad de información, y errores durante el proceso de desarrollo del juego.

Un sistema digital centralizado es necesario para estandarizar, organizar y visualizar estos atributos de manera clara y accesible. Con él, se evita la modificación directa de datos en archivos dispersos, se reducen los errores y se agiliza el flujo de trabajo del equipo. Este sistema transforma una tarea compleja y propensa a fallos en un proceso controlado, eficiente, y fácil de mantener.

Objetivo del Proyecto

El objetivo del proyecto es desarrollar un sistema con interfaz gráfica que permitirá crear, editar y gestionar los atributos esenciales relacionados con NPC. Entre los que se incluyen: **Armas, Habilidades y Clases**.

Este sistema busca centralizar la información, estandarizar su manejo, y facilitar la personalización de personajes, reduciendo errores manuales y optimizando el flujo de trabajo durante el desarrollo del juego. Esto permitirá mantener un control organizado y coherente de los datos, asegurando una integración eficiente con el motor del juego.

Actores que Interactúan con el Sistema

- **Administrador del juego:**

Es el responsable de gestionar toda la información dentro del sistema. Tiene permisos completos para crear, editar y eliminar NPC, así como sus atributos asociados: armas, habilidades, y clases.

- **Desarrollador:**

Utiliza la información registrada en el sistema para integrarla con el motor del juego. Su interacción es principalmente indirecta, ya que consulta y exporta los datos que otros usuarios administran.

Funcionalidades del Sistema

Funciones principales

- **Crear registros:** Permite crear nuevos registros de las entidades de juego: Personajes No Jugadores, Armas, Habilidades y Clases.
- **Editar registros:** Permite modificar los atributos existentes, como nombre, descripción, estadísticas, relaciones con otras entidades, etc.
- **Eliminar o desactivar registros:** permite eliminar definitivamente o desactivar temporalmente entidades que ya no sean necesarias.
- **Visualizar elementos:** Muestra en listas o tablas todas las entidades registradas.
- **Vincular entidades:** Permite relacionar atributos entre sí, por ejemplo, asignar armas a un NPC, asignar habilidades a una clase, asociar clases a personajes.

Funciones opcionales

- **Interfaz gráfica amigable:** Facilita la gestión de las entidades mediante formularios, tablas y navegación intuitiva.
- **Historial de cambios:** mantiene un registro de modificaciones para evitar pérdida de información.
- **Sistema de búsqueda y filtros:** permite localizar rápidamente entidades según nombre, tipo, clase, etc.
- **Control básico de validación:** el sistema verifica que los datos ingresados sean correctos, evitando inconsistencias.

Definir Requerimientos Funcionales y No Funcionales

Requerimientos Funcionales (Qué debe hacer el sistema)

- **CRUD completo:** El sistema debe permitir la **Creación, Lectura, Actualización y Eliminación** de las entidades: Personajes, Armas, Habilidades y Clases.
- **Almacenamiento:** El sistema debe permitir almacenar todos los datos en una base de datos relacional.
- **Interfaz de gestión:** El sistema debe ofrecer una interfaz visual para gestionar la información sin necesidad de código.
- **Validación:** El sistema debe validar la información ingresada para evitar errores y mantener la consistencia.
- Vinculación entre entidades: el sistema debe permitir establecer relaciones entre las entidades, por ejemplo:
    - Asignar armas a personajes
    - Asociar habilidades a clases o personajes
    - Relacionar una clase con sus habilidades correspondientes

Requerimientos No funcionales (Cómo debe funcionar el sistema)

- **Seguridad:** El sistema debe ser seguro, contando con autenticación para asegura que solo usuarios autorizados puedan modificar o eliminar información.
- **Escalabilidad:** El sistema debe ser escalable, permitiendo añadir nuevos tipos de atributos y entidades a futuro sin requerir rehacer todo el sistema.

- **Usabilidad:** La interfaz debe ser intuitiva y consistente para reducir la curva de aprendizaje para los usuarios.
- **Rendimiento:** El sistema debe ofrecer un rendimiento aceptable, sin tiempos de espera excesivos, en la carga y gestión de los datos.

Diagrama Base de datos

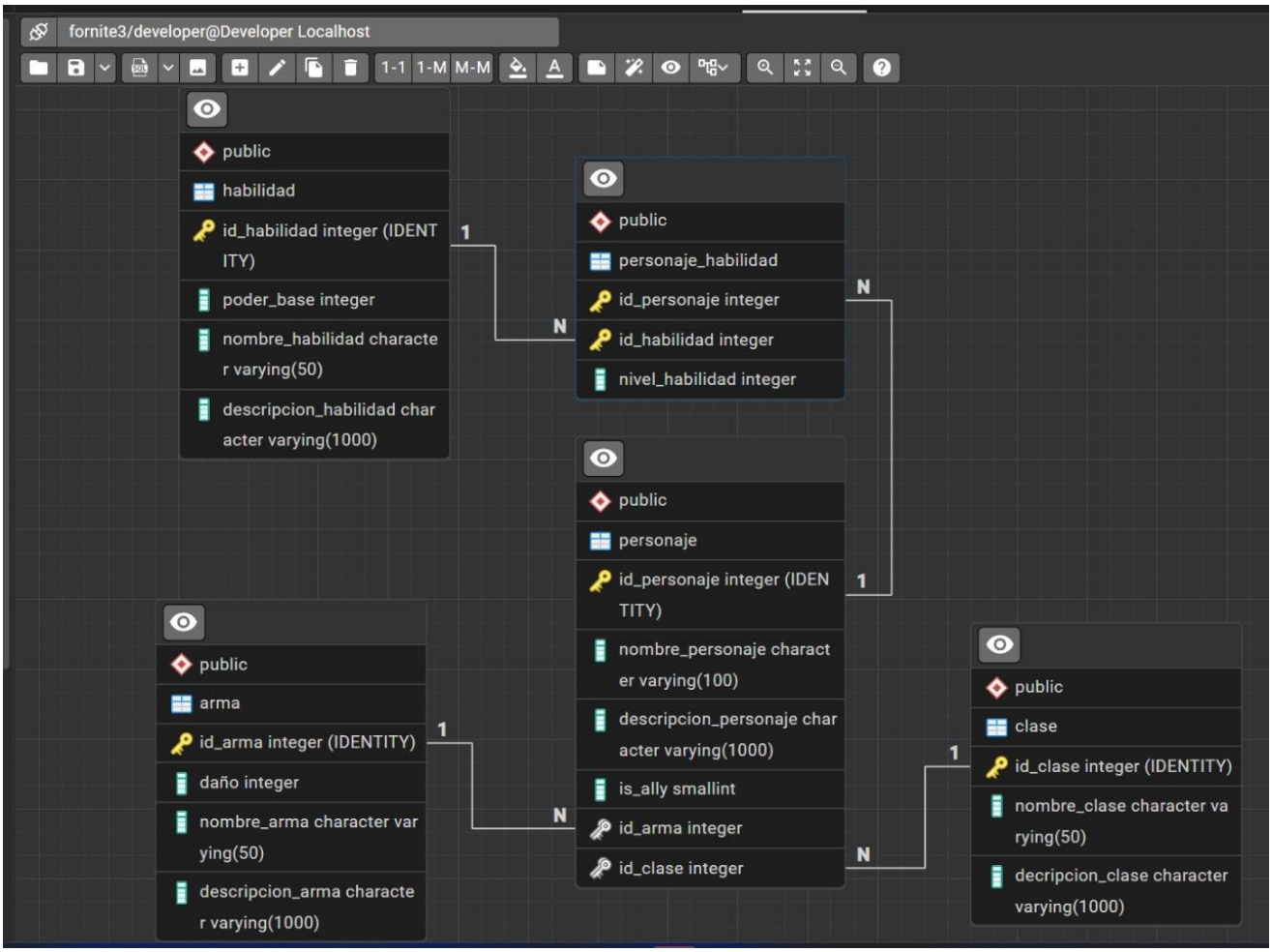
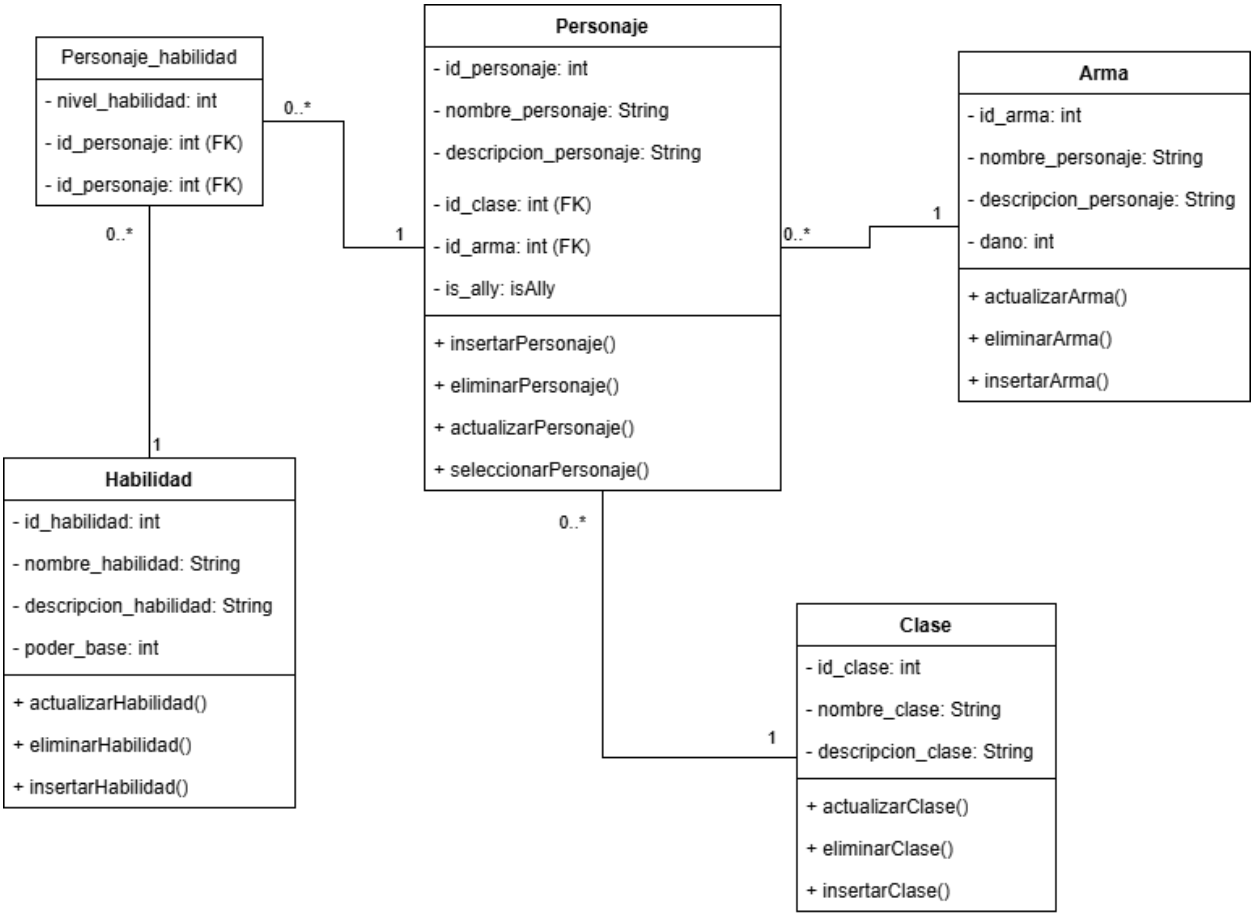


Diagrama UML



Diseño de las vistas

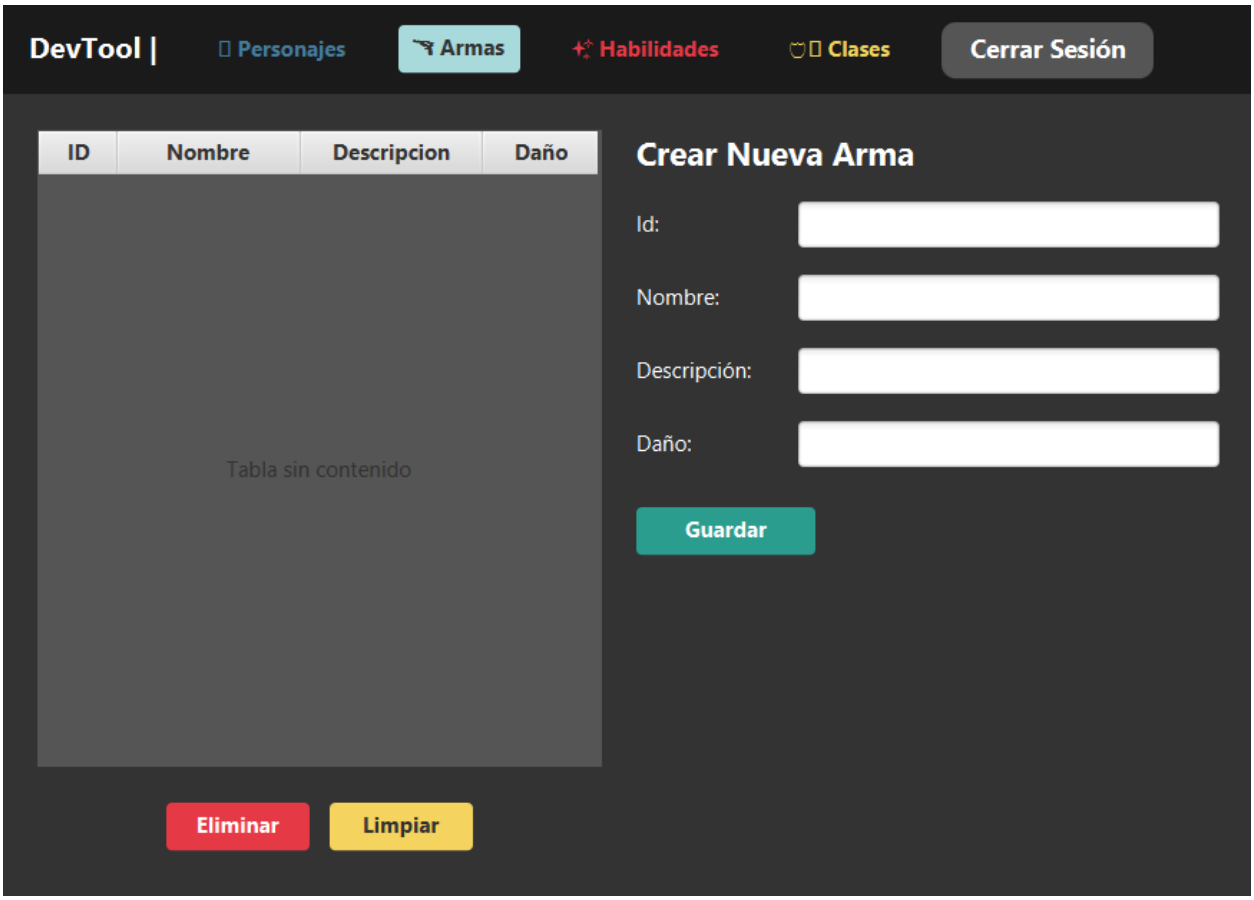
Ventana login



Ventana de habilidades



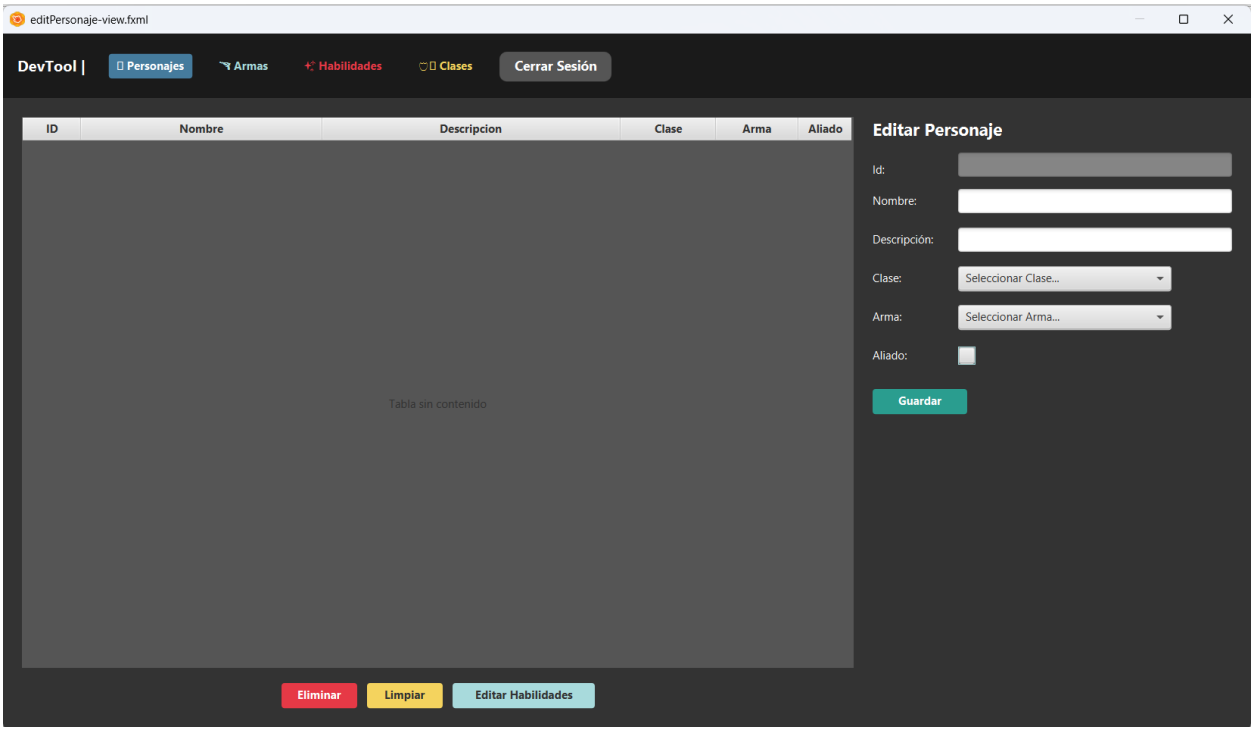
Ventana de armas



Ventana de clases



Ventana personaje



Ventana que relaciona nivel de la habilidad y el personaje

